

DESARROLLO DE SOFTWARE

Trabajo Práctico Integrador: Estructuras de Datos, Contrato
Equals/HashCode y Complejidad Algorítmica

- | | |
|------------------------|-------|
| • Joaquín Echenagorría | 51964 |
| • Ulises Lucato | 52021 |
| • Joaquín Cortez | 51948 |
| • Octavio Crescini | 51949 |
| • Lautaro De Coninck | 51954 |
| • Francisco Valles | 52147 |

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

Traza del Estado Interno:

Explique paso a paso qué ocurre dentro de la estructura HashSet:

- Al ejecutar `tarjetasValidas.add(t1)`.
- Al ejecutar `t1.setNivelAcceso(5)`.
- Al ejecutar `tarjetasValidas.contains(t1)`.

Se ejecuta:

```
TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);
```

Entonces tenemos $\rightarrow t1 \rightarrow \text{codigo} = \text{"ACC-101"} ; \text{nivelAcceso} = 2$

Luego se ejecuta:

```
tarjetasValidas.add(t1);
```

Java utiliza el `hashCode()`, que depende de `"Objects.hash(código, nivelAcceso)"`

Lo que calcula el hash usando el "código (ACC-101)" + "2". El hash determina el bucket donde se almacena **t1**

ACC-101 + 2 $\rightarrow$ hashCode() $\rightarrow$ Bucket X $\rightarrow$ t1

Ahora ocurre una modificación

```
t1.setNivelAcceso(5);
```

Entonces tenemos $\rightarrow t1 \rightarrow \text{codigo} = \text{"ACC-101"} ; \text{nivelAcceso} = 5$

Como tenemos un cambio en el `nivelAcceso` que participa en el `HashCode`, el Hash de `t1` cambia, pero el `HashSet` no vuelve a acomodar el objeto automáticamente en su nuevo bucket por lo que quedaria todavia en su bucket original (Bucket X)

Ahora se ejecuta:

```
tarjetasValidas.contains(t1);
```

Java vuelve a calcular el Hash

ACC-101 + 5 $\rightarrow$ hashCode() $\rightarrow$ Bucket Y $\rightarrow$ t1}

Pero `t1` sigue en el Bucket X por lo tanto `tarjetasValidas.contains(t1)` $\rightarrow$ **false** aunque el objeto sigue almacenado en el Hash.

- Evidencia de ejecución

```
cciones_Colecciones/bin/ControlAcceso
¿Tarjeta válida en sistema?: false
Cantidad de tarjetas registradas: 1
○ joaco.cortezb@MacBook-Air-de-Joaquin Metodos-Colecciones %
```

Explicación del Cajón (Bucket):

Cuando se ejecuta **tarjetasValidas.add(t1)**, el objeto posee los valores **codigo = "ACC-101"** y **nivelAcceso = 2**.

El método **hashCode()** utiliza ambos atributos mediante:

```
Objects.hash(codigo, nivelAcceso);
```

Para estos valores se obtiene un **hashCode** de -1387020803. El **HashSet**, que internamente utiliza un **HashMap**, procesa este hash para determinar la posición dentro de su tabla. Con la capacidad inicial de 16 posiciones utilizada en este caso, el objeto queda almacenado en el bucket 14.

ACC-101 + 2 → hashCode() → Bucket 14 → t1

Luego se ejecuta:

```
t1.setNivelAcceso(5);
```

Como **nivelAcceso** participa en el cálculo de **hashCode()**, el hash del objeto cambia. Para **codigo = "ACC-101"** y , el **nivelAcceso= 5** nuevo **hashCode** es -1387020800, que conduce al bucket 3.

Sin embargo, el **HashSet** no detecta automáticamente esta modificación ni reubica el objeto. Por lo tanto, **t1** continúa almacenado en el bucket 14.

Al ejecutar:

```
tarjetasValidas.contains(t1);
```

Java calcula el hash utilizando el estado actual del objeto y lo busca en el bucket 3. Como **t1** continúa almacenado en el bucket 14, no lo encuentra y **contains(t1)** devuelve **false**.

Bucket 3 → Java busca t1 aquí → no lo encuentra

Bucket 14 → t1 continúa almacenado aquí

Esto demuestra por qué modificar un atributo que participa en **hashCode()** después de insertar un objeto en un **HashSet** puede dejar la estructura en un estado inconsistente.

CÓDIGO TEMPORAL PARA SABER EL NÚMERO EXACTO

```
int h1 = t1.hashCode();
int hashInterno1 = h1 ^ (h1 >>> 16);
int bucket1 = (16 - 1) & hashInterno1;

System.out.println("Hash inicial: " + h1);
System.out.println("Bucket inicial: " + bucket1);

t1.setNivelAcceso(5);

int h2 = t1.hashCode();
int hashInterno2 = h2 ^ (h2 >>> 16);
int bucket2 = (16 - 1) & hashInterno2;

System.out.println("Hash nuevo: " + h2);
System.out.println("Bucket nuevo: " + bucket2);
```

```
...
Bucket inicial: 14
Hash nuevo: -1387020800
Bucket donde Java lo buscaría ahora: 3
¿Tarjeta válida en sistema?: false
Cantidad de tarjetas registradas: 1
○ joaco.cortezb@MacBook-Air-de-Joaquin Metodos-Colecciones %
```

Fuga de Memoria (Memory Leak):

Al modificar **nivelAcceso**, cambia el **hashCode()** del objeto, pero este continúa almacenado en el bucket correspondiente a su hash anterior.

Esto no solo afecta a **contains()**, sino también a operaciones como **remove()**. Si posteriormente se intenta eliminar la tarjeta utilizando su estado actual, el **HashSet** calcula el nuevo hash y la busca en un bucket distinto al que contiene realmente el objeto, por lo que puede no encontrarla.

Si este comportamiento se repite continuamente en un servidor que funciona 24/7, pueden quedar objetos almacenados en el **HashSet** que la aplicación ya no necesita

pero que no logra eliminar correctamente. Como el **HashSet** todavía mantiene una referencia a esos objetos, el Garbage Collector no puede liberarlos.

De esta forma se puede producir una fuga lógica de memoria, haciendo que la cantidad de objetos retenidos y el consumo de memoria aumenten progresivamente.

t1 se agrega al HashSet

Bucket A → t1

Se modifica nivelAcceso

cambia hashCode()

remove(t1)

busca en Bucket B

no encuentra t1

Bucket A → t1 sigue almacenado

El Garbage Collector no puede borrar el objeto porque el HashSet todavía mantiene una referencia a él.

Refactorización Obligatoria:

SOLUCIÓN 1 — INMUTABILIDAD

En la clase original tenemos:

```
private String codigo;  
private int nivelAcceso;
```

y tenemos un setter:

```
public void setNivelAcceso(int nivelAcceso) {  
    this.nivelAcceso = nivelAcceso;  
}
```

El problema es que **nivelAcceso** participa en **equals()** y **hashCode()**, pero puede modificarse después de insertar el objeto en el **HashSet**.

Cambiamos:

```
private String codigo;  
private int nivelAcceso;
```

por:

```
private final String codigo;  
private final int nivelAcceso;
```

El modificador final hace que estos valores solamente puedan asignarse al crear el objeto y después no puedan modificarse.

Ahora tenemos que eliminar:

```
public void setNivelAcceso(int nivelAcceso) {  
    this.nivelAcceso = nivelAcceso;  
}
```

De esta manera ya no podemos hacer → **t1.setNivelAcceso(5);**. Esto evita que cambie un atributo que participa en el cálculo del hashCode().

Ahora mantenemos equals() y hashCode()

```
@Override  
public boolean equals(Object o) {  
    if (this == o) return true;  
    if (o == null || getClass() != o.getClass()) return false;  
  
    TarjetaAcceso tarjeta = (TarjetaAcceso) o;  
  
    return nivelAcceso == tarjeta.nivelAcceso  
        && Objects.equals(codigo, tarjeta.codigo);  
}
```

```
@Override  
public int hashCode() {  
    return Objects.hash(codigo, nivelAcceso);  
}
```

El resultado final nos quedaría que al usar **TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);** el objeto queda con **codigo = "ACC-101"** y **nivelAcceso = 2**; estos valores no pueden modificarse después de crear el objeto, por lo tanto, el **HashCode** no cambia mientras el objeto esté dentro del **HashSet**.

SOLUCIÓN 2 — AJUSTAR EL CRITERIO DE IDENTIDAD

En esta solución consideramos que **codigo** es suficiente para identificar una tarjeta.

Actualmente tenemos:

```
return nivelAcceso == tarjeta.nivelAcceso
    && Objects.equals(codigo, tarjeta.codigo);
```

Lo cambiamos por:

```
return Objects.equals(codigo, tarjeta.codigo);
```

Ahora dos tarjetas se consideran iguales si tienen el mismo código

Modificar el **HashCode**

```
return Objects.hash(codigo, nivelAcceso);
```

Lo modificamos por:

```
return Objects.hash(codigo);
```

Mantener el setter

```
public void setNivelAcceso(int nivelAcceso) {
    this.nivelAcceso = nivelAcceso;
}
```

Podemos seguir haciendo **t1.setNivelAcceso(5);** ya que **nivelAcceso** no participa en el **HashCode**.

Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Requerimiento	Interfaz seleccionada	Implementación concreta	Complejidad búsqueda	Complejidad inserción	Justificación
1. Auditoría	List	ArrayList	$O(n)$	$O(1)$ amortizado	1
2. Ofertas	Set	TreeSet	$O(\log n)$	$O(\log n)$	2
3. Sesiones	Map	HashMap	$O(1)$ promedio	$O(1)$ promedio	3

Justificación 1: Se selecciona List con ArrayList porque permite conservar el orden exacto de inserción y agregar nuevos registros al final con complejidad $O(1)$ amortizada. El recorrido secuencial es $O(n)$. Una búsqueda por elemento es $O(n)$, pero el requerimiento principal no exige búsquedas rápidas sino registrar y recorrer las transacciones en orden.

```
1  import java.util.ArrayList;
2  import java.util.Date;
3  import java.util.List;
4
5  class Transaccion { 3 usages
6      String nombre; 2 usages
7      Date fecha; 2 usages
8
9
10     @Override
11     public String toString(){
12         return "{" + this.nombre + " - " + this.fecha + "}";
13     }
14
15     public Transaccion(String nombre) { 2 usages
16         this.nombre = nombre;
17         this.fecha = new Date();
18     }
19 };
20
21 public class Ecommerce {
22     public static void main(String[] args){
23         List<Transaccion> auditoria = new ArrayList<>();
24
25         auditoria.add(new Transaccion( nombre: "Lionel Messi"));
26         auditoria.add(new Transaccion( nombre: "Diego Maradona"));
27
28         System.out.println(auditoria.toString());
29     }
30 }
31 }
```

Justificación 2: Se utiliza TreeSet porque mantiene automáticamente los productos ordenados. Su estructura de árbol balanceado permite insertar elementos y realizar operaciones de búsqueda en $O(\log n)$, además de acceder eficientemente al producto de menor y mayor precio mediante los extremos del conjunto.

Justificación 3: Se utiliza un HashMap porque cada token es único y puede utilizarse como clave asociada a una sesión. Tanto la inserción mediante `put()` como la búsqueda mediante `get()` tienen complejidad $O(1)$ promedio, lo que resulta especialmente adecuado para buscar entre aproximadamente 500.000 usuarios activos.

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

```
public static Character primerCaracterNoRepetido(String texto) {  
    Map<Character, Integer> frecuencias = new HashMap<>();  
  
    // Primer recorrido: contar apariciones  
    for (char c : texto.toCharArray()) {  
        frecuencias.put(c, frecuencias.getOrDefault(c, default: 0) + 1);  
    }  
  
    // Segundo recorrido: encontrar el primero con frecuencia 1  
    for (char c : texto.toCharArray()) {  
        if (frecuencias.get(c) == 1) {  
            return c;  
        }  
    }  
  
    return null;  
}
```

Se utiliza un **HashMap<Character, Integer>** para almacenar cada carácter junto con la cantidad de veces que aparece en el texto.

En un primer recorrido se cuentan las apariciones de cada carácter. Las operaciones **get()** y **put()** de **HashMap** tienen complejidad promedio $O(1)$, por lo que recorrer una cadena de **n** caracteres cuesta $O(n)$.

Luego se realiza un segundo recorrido lineal sobre el texto original para encontrar el primer carácter cuya frecuencia sea igual a 1.

Como ambos recorridos son consecutivos y no están anidados, la complejidad total es:

$$O(n) + O(n) = O(n)$$

De esta forma se mejora el algoritmo original, que tenía complejidad $O(n^2)$ debido al uso de dos ciclos **for** anidados.

ANÁLISIS DE COLISIONES: Una colisión de hash ocurre cuando dos claves diferentes generan el mismo `hashCode()`.

En ese caso, **HashMap** puede ubicar ambas claves en el mismo bucket, pero no pierde ninguna de ellas. Para distinguirlas, además del `hashCode()`, Java utiliza el método `.equals()`

Primero se utiliza el hash para determinar el bucket donde debe buscarse la clave. Si dentro de ese bucket existen varias claves debido a una colisión, se comparan mediante `equals()` para identificar cuál es la correcta.

Por lo tanto, dos claves pueden tener el mismo `hashCode()` y seguir almacenándose de manera independiente siempre que `equals()` determine que son diferentes.

Las colisiones pueden reducir el rendimiento si se acumulan muchas claves en un mismo bucket, pero no provocan pérdida de información.

CONSIGNA 4:

- 1) El **HashSet<String>** contendrá **2 elementos**. Aunque **"FB"** y **"Ea"** generan el mismo `hashCode()` (2236), esto solamente provoca una colisión de hash y no significa que los objetos sean iguales.

El **HashSet** utiliza primero `hashCode()` para determinar el bucket donde debe almacenarse el elemento. Como ambas cadenas generan el mismo hash, pueden ubicarse en el mismo bucket. Luego Java utiliza `equals()` para comprobar si realmente representan el mismo elemento.

Al evaluar `"FB".equals("Ea")`, el resultado es **false**, por lo que el **HashSet** determina que son dos cadenas distintas y almacena ambas.

- 2) En condiciones normales, las operaciones de búsqueda e inserción de un **HashMap** tienen una complejidad promedio de $O(1)$, ya que el hash permite localizar rápidamente el bucket correspondiente.

Sin embargo, si se acumulan muchas colisiones y varios elementos terminan en el mismo bucket, Java debe realizar comparaciones adicionales dentro de ese bucket.

Si los elementos se encuentran organizados de forma lineal, el peor caso puede degradarse hasta **$O(n)$** , ya que puede ser necesario recorrer todos los elementos del bucket.

En implementaciones modernas de Java, cuando se acumula una cantidad suficiente de elementos en un mismo bucket, la estructura puede transformarse en un árbol balanceado. En ese caso, las operaciones pueden mejorar a aproximadamente **$O(\log n)$** .

Por lo tanto, las colisiones no provocan pérdida de información, pero una gran cantidad de ellas puede afectar negativamente el rendimiento de la estructura.

